JOHANNES KEPLER
UNIVERSITY LINZ

# Hardware Description with SystemVerilog



Simon Dorrer

**KV VLSI Design, 336.051, WiSe 26/27**

IICQC
INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

**JOHANNES KEPLER**
**UNIVERSITÄT LINZ**
Altenberger Straße 69
4040 Linz, Österreich
jku.at

# Today's Goals

After this lecture, you can

- explain why an HDL describes **hardware, not software**
- describe a circuit as **register + next-state logic**
- name the main **HDL families** and their trade-offs
- translate your **two-process VHDL** into SystemVerilog
- read and modernize plain **Verilog**
- avoid **latches, races, and width errors**
- write self-checking **SystemVerilog** and **cocotb** testbenches
- **lint, simulate, and synthesize** in IIC-OSIC-TOOLS



Chapter 2

Lecture notes: Chapter 2 of [iic-jku.github.io/vlsi-design](iic-jku.github.io/vlsi-design)

# Recap: Hardware, Not Software



# MCU Software vs. Hardware Description

| | Software for an MCU (C) | Hardware description (HDL) |
|---|---|---|
| What is described | instructions for a fixed processor | the circuit itself |
| Execution | one instruction after another | all blocks concurrently, all the time |
| Time | implicit, follows from the instruction count | explicit, registers update at clock edges |
| A variable is | a memory location or CPU register | a wire or a flip-flop |
| A loop is | a repetition in time | a repetition in space (unrolled) |
| A function call is | a jump to shared code | a separate copy of the hardware |
| Cost | memory and clock cycles | area, power, maximum clock frequency |
| Fixing a bug | a firmware update | for an ASIC, a new mask set |

**Think hardware first, then write the code.**

# Computing in Time and in Space

### C on an MCU

```
uint8_t popcount(uint8_t x)
{
  uint8_t count = 0;
  for (int i = 0; i < 8; i++)
    count += (x >> i) & 1;
  return count;
}
```

- How many **clock cycles**?
- What happens with **64 bits**?

### SystemVerilog

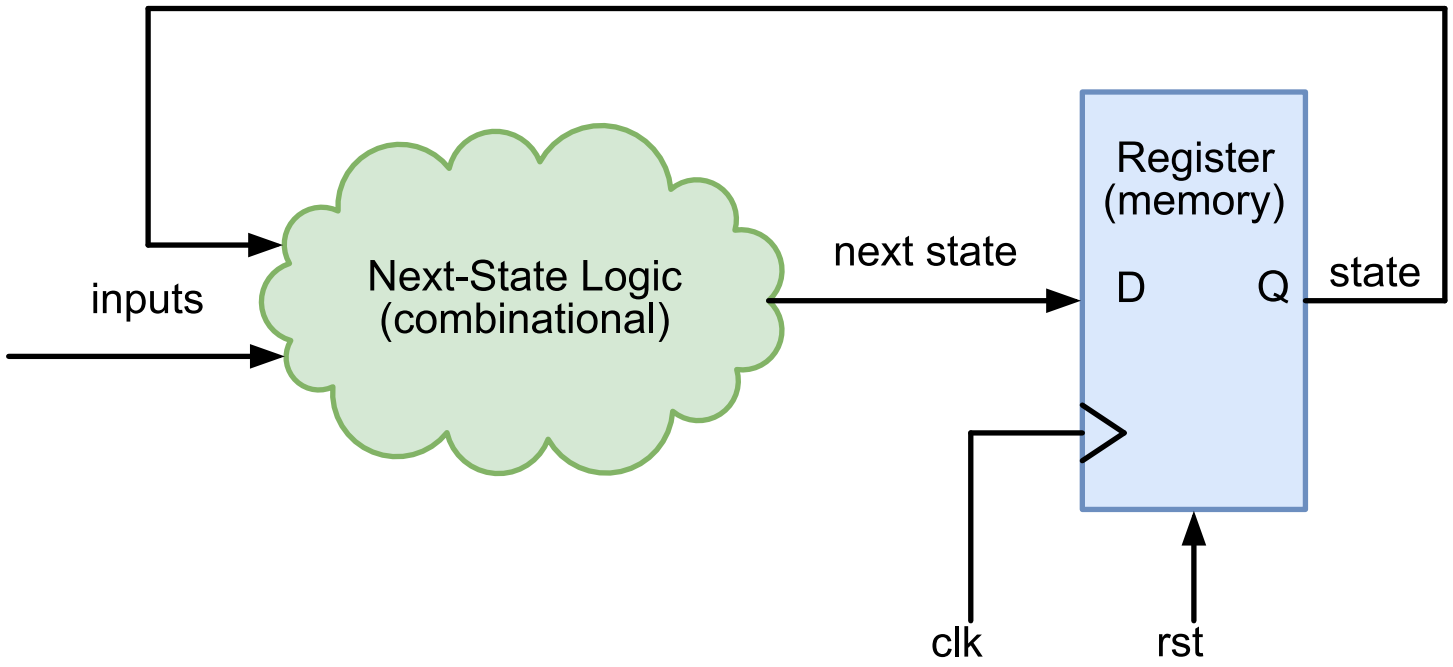
```
module popcount (
  input  logic [7:0] x_i,
  output logic [3:0] count_o
);
  always_comb begin
    count_o = '0;
    for (int i = 0; i < 8; i++) begin
      count_o = count_o + 4'(x_i[i]);
    end
  end
endmodule
```

- What does synthesis build from the **loop**?

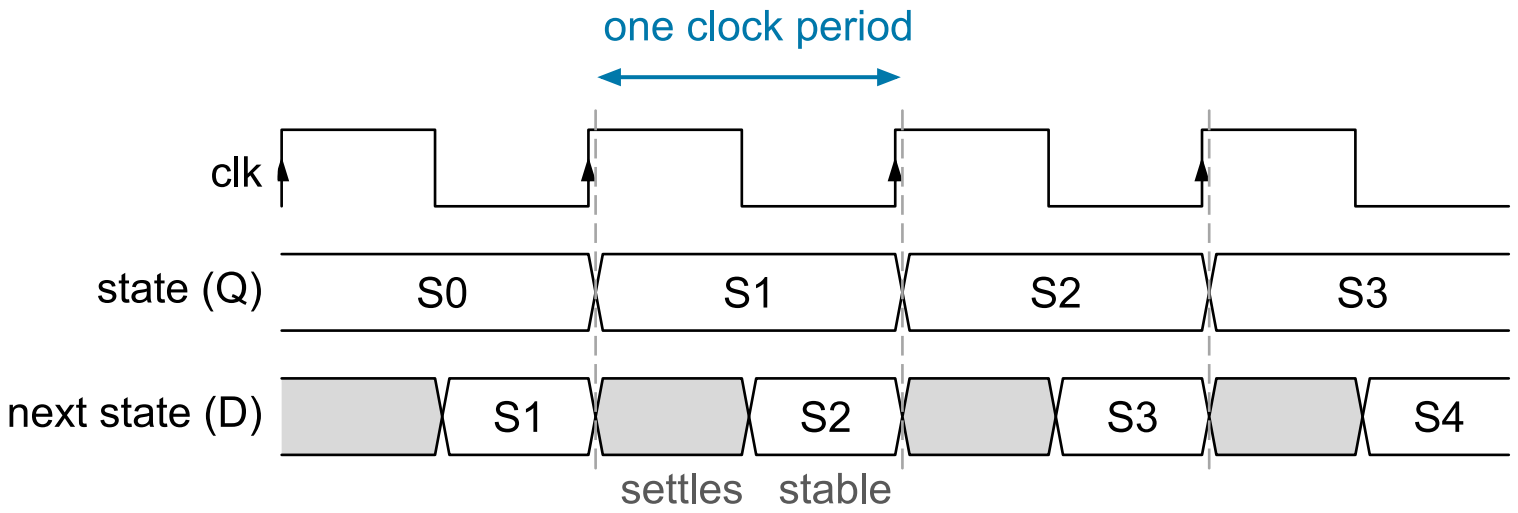
**Exercise** (Section 2.2.1): the three questions, then synthesize `popcount.sv`.

# Synchronous Sequential Design

- **Register**: edge-triggered flip-flops store the state
- **Next-state logic**: combinational, from state and inputs
- **Rising edge**: the register loads the next state
- Outputs from the state (**Moore**) or also from the inputs (**Mealy**)



# One Clock Period to Compute




- Flip-flop outputs change **only at the rising edge**
- The logic sees **stable inputs** and has **one period** to compute, minus the setup time
- Clock period > slowest path: checked by **static timing analysis** (Chapter 5)
- Asynchronous inputs need a **synchronizer** first (Chapter 9)

# The Two-Process Style in VHDL

```vhdl
-- Register process: only reset and clock
reg_proc: process(rst_i, clk_i) is
begin
  if rst_i = '1' then
    state <= '0';
  elsif rising_edge(clk_i) then
    state <= next_state;
  end if;
end process reg_proc;

-- Combinational process: all signals read
next_state_logic: process(state, toggle_i) is
begin
  -- default assignment
  next_state <= state;
  -- next_state from state and inputs
  if toggle_i = '1' then
    next_state <= not state;
  end if;
end process next_state_logic;
```

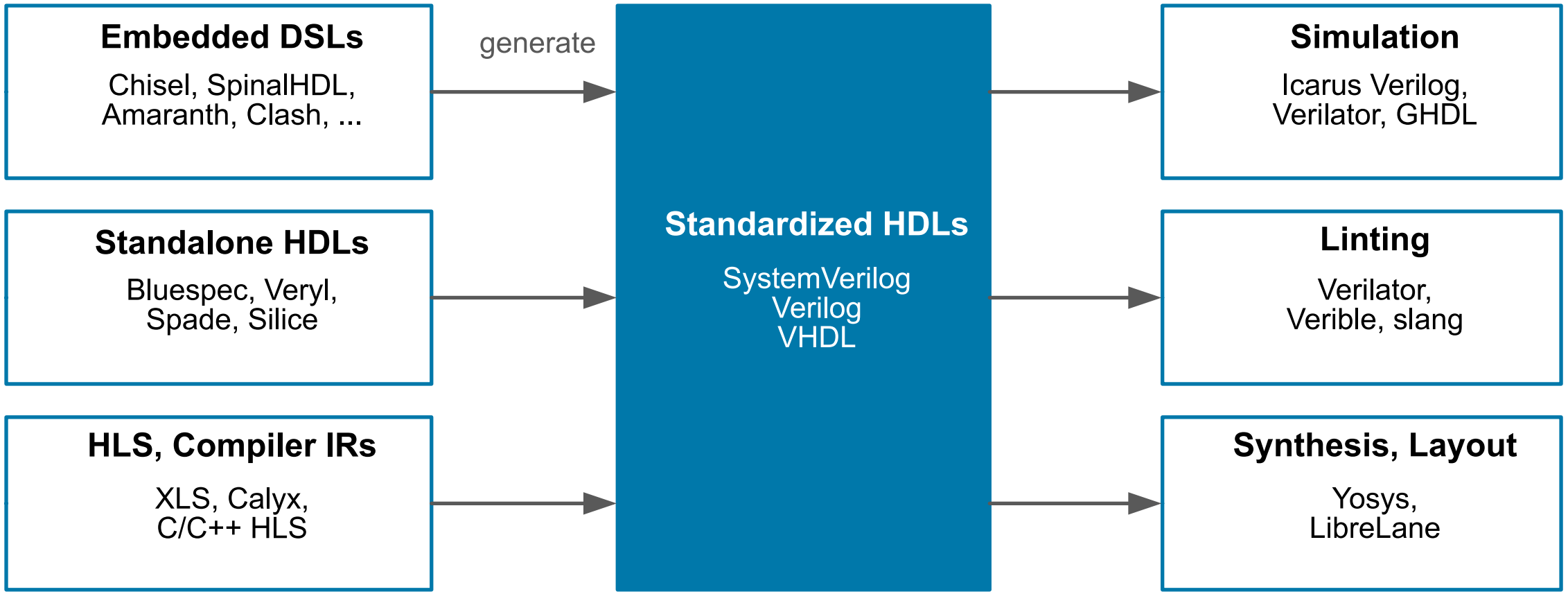
Three rules:

- **Register process**: only **reset and clock** in the sensitivity list, no logic
- **Combinational process**: **all signals it reads** in the sensitivity list (VHDL-2008: `process(all)`)
- **Default assignment first**, then all decisions: no latch

# The HDL Landscape



# Four Families of HDLs




All alternatives generate **(System)Verilog**, the exchange format of the industry

# Alternative HDLs at a Glance

| Family | Host language | Languages (selected origins) |
|---|---|---|
| Embedded DSL | Scala | Chisel (UC Berkeley), SpinalHDL, DFiant |
| Embedded DSL | Python | Amaranth, Migen (M-Labs), PyMTL3 (Cornell), PyRTL, HWT, Magma and Kratos (inactive) |
| Embedded DSL | Haskell, OCaml | Clash (University of Twente), Hardcaml (Jane Street) |
| Embedded DSL | Dart, Rust | ROHD (Intel), RHDL |
| Standalone HDL | own syntax | Bluespec, Veryl, Spade, Silice |
| HLS, compiler IR | DSLX, C/C++, IR | XLS (Google), Calyx (Cornell), Vitis HLS, Catapult, Bambu |

# Comparing the Families

| | Standardized HDLs | Embedded DSLs | Standalone HDLs | HLS |
|---|---|---|---|---|
| Adoption | very high, all EDA tools, IP, jobs | niche (Rocket Chip, VexRiscv) | research, early adopters | data paths in commercial flows |
| Tool support | every tool | host toolchain + generated Verilog | one compiler | mostly commercial |
| Productivity | RTL, limited generators | high: generators | medium to high | highest: algorithms |
| Error checking | VHDL strongly typed, SV needs lint | host type system | strong types (Bluespec: no races) | timing implicit |
| Debugging | names match | generated names | like embedded DSLs | RTL hard to relate |
| Long-term risk | decades of stability | small projects, some dormant | early stage | vendor lock-in |

# Why SystemVerilog in This Course?

- **Industry standard** (IEEE 1800-2023): every EDA tool, IP vendor, and foundry flow
- **Open-source tools**: Verilator, Icarus Verilog, Yosys, LibreLane, slang, Verible, cocotb
- **You have to read it anyway**: netlists, gate-level simulations, generated code
- **cocotb for verification**: testbenches in Python, SystemVerilog for the RTL
- **A good foundation**: Chisel or Amaranth is quick to learn later

# VHDL vs. SystemVerilog

| Aspect | VHDL | SystemVerilog |
|---|---|---|
| Standard | IEEE 1076-2019 | IEEE 1800-2023 (includes Verilog) |
| Typing | strong, explicit conversions | weak, silent extension and truncation |
| Verbosity | verbose | compact |
| Case | case-insensitive | case-sensitive (`clk` and `Clk` differ) |
| Semantics | delta cycles, order never matters | races if coding rules are broken |
| Verification | libraries (OSVVM, UVVM) | built in: classes, SVA, coverage |
| Typical use | FPGA, aerospace, Europe, academia | ASIC design and verification worldwide |
| Open-source tools | GHDL | Verilator, Icarus Verilog, Yosys, slang |

VHDL protects you with types. SystemVerilog relies on **discipline and the linter**.

# Cheat-Sheets and Guidelines

- **SystemVerilog** and **Verilog cheat-sheets** in the AMS chip template
  - `doc/verilog`: syntax references, not style guides, lint what you copy
- **Circuit Designer's Etiquette**: VHDL/Verilog Coding Guide
- **lowRISC Verilog Coding Style Guide**: most of our naming
- **IEEE 1800-2023**: free through the IEEE GET Program
- Books:
  - Harris and Harris: SystemVerilog and VHDL side by side
  - Sutherland: RTL modeling with SystemVerilog
  - Sutherland and Mills: 101 gotchas

Cheat-Sheets

Etiquette

# SystemVerilog for VHDL Designers



# VHDL and SystemVerilog at a Glance

| Concept | VHDL | SystemVerilog |
|---|---|---|
| Design unit | `entity` + `architecture` | `module ... endmodule` |
| Parameter | `WIDTH : positive := 8` | `parameter int unsigned WIDTH = 8` |
| Port | `a_i : in std_logic_vector(7 downto 0)` | `input logic [7:0] a_i` |
| Bit, vector | `std_logic`, `std_logic_vector` | `logic`, `logic [N-1:0]` |
| All zeros | `(others => '0')` | `'0` |
| Concatenation | `a & b` | `{a, b}` |
| Conditional | `y <= a when s = '1' else b;` | `assign y = s ? a : b;` |
| Combinational | `process(a, b)` | `always_comb` |
| Clocked | `process(rst_i, clk_i)` + `rising_edge` | `always_ff @(posedge clk_i or negedge rst_ni)` |
| Enumeration | `type t_state is (IDLE, RUN);` | `typedef enum logic {StIdle, StRun} state_e;` |
| Instance | `u0: entity work.adder port map(...)` | `adder u0 (...);` |
| Width of N | `natural(ceil(log2(real(N))))` | `$clog2(N)` |

# Modules, Ports, and Parameters

## VHDL

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity adder is
  generic(
    WIDTH : positive := 8
  );
  port(
    a_i   : in  std_logic_vector(WIDTH - 1 downto 0);
    b_i   : in  std_logic_vector(WIDTH - 1 downto 0);
    sum_o : out std_logic_vector(WIDTH downto 0)
  );
end entity adder;
architecture rtl of adder is
begin
  sum_o <= std_logic_vector(
    resize(unsigned(a_i), WIDTH + 1)
    + unsigned(b_i));
end architecture rtl;
```

## SystemVerilog

```systemverilog
`default_nettype none
module adder #(
  parameter int unsigned WIDTH = 8
) (
  input  logic [WIDTH-1:0] a_i,
  input  logic [WIDTH-1:0] b_i,
  output logic [WIDTH:0]   sum_o
);
  assign sum_o = {1'b0, a_i} + {1'b0, b_i};
endmodule
`default_nettype wire
```

- No type conversions
- `` `default_nettype none ``
  - a typo becomes an error
- Case-sensitive, one module per file

# Data Types: logic

- `logic`: four values `0`, `1`, `x`, `z`, for **every RTL signal**
- **One driver** per variable (Verilator: `MULTIDRIVEN`)
- `reg` is **not a register**: storage follows from how a variable is assigned
- Vectors: `logic [7:0] data`, `data[7:4]`, `data[i*4 +: 4]`
- Unsigned by default, `logic signed [15:0]`: **do not mix** them
- Two-state `bit`, `int`: parameters, loop variables, testbenches only
- Icarus is four-state (x visible), Verilator two-state: **simulate with both**

# Literals

| VHDL | SystemVerilog | Meaning |
|---|---|---|
| `"10100101"` | `8'b1010_0101` | 8-bit binary, _ for readability |
| `x"A5"` | `8'hA5` | 8-bit hexadecimal |
| `std_logic_vector(to_unsigned(165, 8))` | `8'd165` | 8-bit decimal |
| `(others => '0')`, `(others => '1')` | `'0`, `'1` | all bits zero or one, any width |
| `(others => 'X')`, `(others => 'Z')` | `'x`, `'z` | all bits unknown or high impedance |
| `"10X1"` | `4'b10x1` | a literal with an unknown bit |
| `165` | `165` | unsized: a 32-bit signed integer |

**Always size your literals.**

# Operators

| VHDL | SystemVerilog | Meaning |
|---|---|---|
| `not a` | `~a` | bitwise NOT |
| `a and b`, `a or b`, `a xor b` | `a & b`, `a \| b`, `a ^ b` | bitwise AND, OR, XOR |
| `and a` (VHDL-2008) | `&a`, `\|a`, `^a` | reduction, 1-bit result |
| `not`, `and`, `or` on `boolean` | `!a`, `a && b`, `a \|\| b` | logical, 1-bit result |
| `=`, `/=` | `==`, `!=` | equality, can be x with x operands |
| `<`, `<=`, `>`, `>=` | `<`, `<=`, `>`, `>=` | relational |
| none | `===`, `!==` | case equality, testbenches only |
| `shift_left()`, `shift_right()` | `<<`, `>>`, `>>>` | logical, arithmetic shift |
| `a & b` | `{a, b}`, `{4{a}}` | concatenation, replication |
| `a when s = '1' else b` | `s ? a : b` | conditional operator |

# Expression Widths

```
logic [7:0] a, b, sum8;
logic [8:0] sum9;
logic [3:0] nibble, count;
logic       hit;

assign sum8   = a + b;          // carry silently lost, no warning
assign sum9   = a + b;          // carry kept, the target is 9 bits wide
assign nibble = a;              // MSBs cut off (Verilator: WIDTHTRUNC)
assign hit    = (count == 20);  // always false (Verilator: WIDTHEXPAND)
```

- Operands extend to the widest operand **and the target**, then truncate: **no error**
- Verilator catches the truncation and the impossible compare, **not the lost carry**
- Make extensions explicit: `{1'b0, a}` or `9'(a)`

# Concurrent Assignments

```
assign y_o   = a_i & b_i;           // VHDL: y_o <= a_i and b_i;
assign mux_o = sel_i ? a_i : b_i;   // VHDL: mux_o <= a_i when sel_i = '1' else b_i;
```

- `assign` is re-evaluated whenever an operand changes
- `with ... select` becomes a `case` in `always_comb`
- Output ports can be **read** inside the module: no internal copy needed

**Exercise** (Section 2.5.4): translate the VHDL 4:1 multiplexer `mux4` into SystemVerilog and lint it with `verilator --lint-only -Wall`.

# Processes

- `always_comb`: combinational logic
  - complete sensitivity list inferred, like `process(all)`, the tools check for latches
- `always_ff @(posedge clk_i or negedge rst_ni)`: flip-flops
  - only the clock edge and the reset edge in the event list
- `always_latch`: an intended latch (rarely needed)
- Two assignments:
  - **blocking** (=): updates immediately, like a VHDL variable `:=`
  - **non-blocking** (<=): updates after all blocks have read their inputs, like a VHDL signal

# Blocking vs. Non-Blocking

```
always_ff @(posedge clk_i) begin
  a_q <= b_q;
  b_q <= a_q;  // reads the old a_q: a real swap
end
```

- With = in this block: b_q reads the **new** a_q, both end up with the old b_q: **no swap**
- With = in **two blocks**: the result depends on the order in which the simulator runs the blocks, synthesis builds the swap: **simulation and hardware differ**

**The Golden Rules**

1. `always_ff`: only `<=`
2. `always_comb`: only `=`
3. One block per variable
4. No plain `always` or `initial` in RTL

# Registers and Resets

```
// Asynchronous active-low reset (course default)
always_ff @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) begin
    q <= '0;
  end else begin
    q <= d;
  end
end

// Synchronous active-high reset (template counter)
always_ff @(posedge clk_i) begin
  if (reset_i) begin
    q <= '0;
  end else begin
    q <= d;
  end
end
```

- Default: async active-low `rst_ni`
- Matches the cells: `sg13cmos5l_dfrbp_1` has an active-low `RESET_B`
- Reset test first, polarity matches the event list
- Assert asynchronously, **release synchronously** (Chapter 9)
- No reset: only `posedge clk_i` in the event list
- Clock enable: a register, **not a latch**

# The Two-Process Style in SystemVerilog

```
// Register process: only clock and reset
always_ff @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) begin
    state_q <= 1'b0;
  end else begin
    state_q <= state_d;
  end
end

// Combinational process: list inferred
always_comb begin
  // default assignment
  state_d = state_q;

  // state_d from state_q and inputs
  if (toggle_i) begin
    state_d = ~state_q;
  end
end
```

- `state` / `next_state` become `state_q` / `state_d` (**Q** and **D**)
- `always_ff`: only clock and reset edge (data signal: Yosys error)
- `always_comb`: sensitivity list **inferred**
- **Default first**, else Verilator `LATCH` and a Yosys error
- Yosys: `$_DFFE_PN0P_` and an inverter, the default became the **enable**

# Example: Tick Generator

**VHDL: `counter_logic`**

```vhdl
counter_logic: process(count, en_i) is
begin
  next_count <= count;
  next_tick <= '0';
  if en_i = '1' then
    if count = C_LAST then
      next_count <= (others => '0');
      next_tick <= '1';
    else
      next_count <= count + 1;
    end if;
  end if;
end process counter_logic;
```

**SystemVerilog: `always_comb`**

```systemverilog
always_comb begin
  count_d = count_q;
  tick_d  = 1'b0;
  if (en_i) begin
    if (count_q == COUNT_LAST) begin
      count_d = '0;
      tick_d  = 1'b1;
    end else begin
      count_d = count_q + 1'b1;
    end
  end
end
```

- `tick_o` is a **strobe**: high for one cycle every `DIV` cycles while `en_i` is high

# if, case, and Latches

```
// Latch: y_o holds its value while en_i is low
always_comb begin
  if (en_i) y_o = a_i;
end

// Fixed: default assignment first
always_comb begin
  y_o = '0;
  if (en_i) y_o = a_i;
end
```

- A `case` need **not** be complete: a missing value means a **latch**
- `unique case` + `default`
- `casez` for decoders, avoid `casex`
- `always @(*)`: Yosys warns and builds a latch
- `always_comb`: Yosys **stops with an error**

# Exercise: Let the Linter Find the Bugs

```
logic [7:0] alu;
logic [7:0] result_q;
// Result register
always_ff @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) begin
    result_q = 8'h00;
  end else begin
    result_q = alu;
  end
end
// ALU: 00 add, 01 subtract, 10 AND, 11 OR
always_comb begin
  case (op_i)
    2'b00: alu = a_i + b_i;
    2'b01: alu = a_i - b_i;
    2'b10: alu = a_i & b_i;
  endcase
end
assign result_o = result_q;
assign zero = (result_q == 8'h00);
```

A small ALU: `op_i` selects add, subtract, AND, or OR. `result_o` is registered, `zero_o` flags a zero result.

- Lint it with Verilator (`-Wall`)
- Explain **every** warning
- Fix it until Verilator is silent
- Synthesize both versions with Yosys

**Exercise** (Section 2.5.8): lint, fix, and synthesize the full file.

# Finite State Machines

```
typedef enum logic [1:0] {StIdle, StRun, StDone} state_e;
state_e state_d, state_q;

always_comb begin
  state_d = state_q;  // defaults first
  busy_o  = 1'b0;
  done_o  = 1'b0;
  unique case (state_q)
    StIdle: if (start_i) state_d = StRun;
    StRun: begin
      busy_o = 1'b1;
      if (last_i) state_d = StDone;
    end
    StDone: begin
      done_o  = 1'b1;
      state_d = StIdle;
    end
    default: state_d = StIdle;  // unused 2'b11
  endcase
end
```

- VHDL: `type t_state is (...)`, the same `fsm_logic` structure
- `typedef enum` with base type, `_e`, `St` prefix
- Unused `2'b11`: `default`
- **Defaults first**: state and outputs
- Transitions with `if`/`else` (Icarus rejects `?:` between enums)
- Yosys re-encodes one-hot

**Exercise** (Section 2.5.9): the controller as a Mealy FSM.

# Hierarchy and Generate

## VHDL

```
adder_0: entity work.adder
  generic map(
    WIDTH => 16
  )
  port map(
    a_i => x,
    b_i => y,
    sum_o => s
  );
```

- Connect **by name**, every port
- No `.*` or `.name`
- Instance name: `gen_lane[2].adder_i`

## SystemVerilog

```
adder #(
  .WIDTH(16)
) adder_0 (
  .a_i  (x),
  .b_i  (y),
  .sum_o(s)
);
```

```
for (genvar i = 0; i < 4; i++) begin : gen_lane
  adder #(
    .WIDTH(8)
  ) adder_i (
    .a_i  (lane_a[i]),
    .b_i  (lane_b[i]),
    .sum_o(lane_sum[i])
  );
end
```

# Packages, Typedefs, and Functions

```
package alu_pkg;
  localparam int unsigned DATA_WIDTH = 8;

  typedef enum logic [1:0] {
    OpAdd, OpSub, OpAnd, OpOr
  } alu_op_e;

  typedef struct packed {
    alu_op_e                  op;
    logic [DATA_WIDTH-1:0] operand;
  } instr_t;

  function automatic logic is_zero(
      input logic [DATA_WIDTH-1:0] data);
    is_zero = (data == '0);
  endfunction
endpackage
```

- Use: `alu_pkg::OpAdd`, `alu_pkg::is_zero(result_o)`
- `struct packed` replaces `record`
- `function automatic`: combinational logic, one copy per call
- Compile the package **first**

**Yosys 0.70 default front end** (LibreLane): no `import alu_pkg::*`, no `return`, no unpacked array ports.

Use `pkg::NAME`, or set `USE_SLANG: True`.

# What Is Synthesizable?

| Synthesizable (RTL) | Simulation and testbenches only |
|---|---|
| `module`, `parameter`, `localparam`, `assign` | `initial` (ASICs have no initial values: use a reset) |
| `always_comb`, `always_ff`, `always_latch` | delays `#10`, event waits `@(...)` and `wait` |
| `if`, `case`, `for` with constant bounds | `$display`, `$finish`, `$fatal`, `$dumpvars` |
| `function automatic`, generate | `===`, `!==`, `real`, `string` |
| `typedef enum`, `struct packed`, packages | classes, `randomize()`, `fork` and `join` |
| `+`, `-`, `*`, shifts, comparisons | loops with data-dependent iterations |

`/` and `%`: large and slow, unless the divisor is a constant power of two

# Verilog vs. SystemVerilog

| Aspect | Plain Verilog | SystemVerilog |
|---|---|---|
| Signals | `wire` for `assign`, `reg` for `always` | `logic` for both |
| Ports | `module m (a, y); input a; ...` | `module m (input logic a_i, ...);` |
| Combinational | `always @(*)`: a latch is a Yosys warning | `always_comb`: a latch is a Yosys **error** |
| Flip-flops | `always @(posedge clk or negedge rst_n)` | `always_ff @(posedge clk_i or negedge rst_ni)` |
| Constants | `` `define ``, a `localparam` per state | `package`, `typedef enum` for the states |
| All zeros | `{W{1'b0}}` | `'0` |
| Loops | `integer i;` … `i = i + 1` | `for (int i = 0; i < N; i++)` |
| Case checks | `// synopsys full_case` comment | `unique case` + `default` |
| Tools | `.v`, `read_verilog`, `iverilog` | `.sv`, `read_verilog -sv`, `iverilog -g2012` |

**Read** Verilog (netlists, cell models, generated code, older IP), **write** SystemVerilog

**Exercise** (Section 2.5.13): modernize a Verilog-2001 priority encoder.

# Testbenches



# A SystemVerilog Testbench

```
always #(CLK_PERIOD_NS / 2) clk = ~clk;

initial begin
  $dumpfile("tick_gen_tb.vcd");
  $dumpvars(0, tick_gen_tb);
  repeat (2) @(negedge clk);
  rst_n = 1'b1;
  // ... 3 * DIV cycles with en = 0: no tick ...
  en = 1'b1;
  for (int cycle = 1; cycle <= 4 * DIV; cycle++) begin
    @(negedge clk);
    if (tick !== (cycle % DIV == 0)) begin
      $fatal(1, "FAIL: cycle %0d, tick = %b", cycle, tick);
    end
  end
  $display("PASS: tick_gen_tb");
  $finish;
end
```

- Module **without ports**, DUT with a small `DIV`
- Drive and check on the **falling edge**
- `!==` also fails on x
- Fail with `$fatal(1, ...)`
  - `$error` exits with 0 in Icarus
- `$dumpvars`: VCD waveform

# A cocotb Testbench

```python
@cocotb.test()
async def test_tick_every_div_cycles(dut):
    """tick_o is high for one cycle every DIV cycles."""
    await start_up(dut)  # 50 MHz clock, reset for two cycles
    dut.en_i.value = 1
    for cycle in range(1, 4 * DIV + 1):
        await FallingEdge(dut.clk_i)
        expected = 1 if cycle % DIV == 0 else 0
        assert dut.tick_o.value == expected, \
            f"cycle {cycle}: expected {expected}, got {dut.tick_o.value}"

# end of the runner: a failing test must set the exit status
num_tests, num_failed = get_results(results_xml)
if num_failed:
    raise SystemExit(f"FAIL: {num_failed} of {num_tests} tests failed")
```

- Tests are coroutines: `await FallingEdge(...)`, `ClockCycles(...)`, `Timer(...)`
- A write takes effect at the **end of the time step**

# SystemVerilog or cocotb?

| Aspect | SystemVerilog testbench | cocotb testbench |
|---|---|---|
| Language | SystemVerilog, the full language | Python 3 |
| Reference models | written in SystemVerilog | any Python code and library (NumPy) |
| Simulators | every SystemVerilog simulator | Icarus, Verilator, GHDL, commercial |
| DUT languages | SystemVerilog, VHDL only with a mixed-language simulator | VHDL and SystemVerilog alike |
| Speed | fast, all in the simulator | slower, every access goes through Python |
| Template folder | `testbenches/verilog/` | `testbenches/cocotb/` |

Both: SystemVerilog for a **quick self-check**, cocotb for **models and random stimuli**

# Lint, Simulate, and Synthesize



# Linting with Verilator

```
verilator --lint-only -Wall rtl/tick_gen.sv
```

| Warning | Meaning | Typical fix |
|---|---|---|
| `IMPLICIT` | undeclared name, an implicit 1-bit wire | fix the typo, `` `default_nettype none `` |
| `UNDRIVEN`, `UNUSEDSIGNAL` | never assigned or never read | connect or remove it |
| `WIDTHTRUNC`, `WIDTHEXPAND` | widths do not match | sized literals, `{1'b0, x}`, `N'(x)` |
| `LATCH` | a combinational block misses a path | defaults first |
| `CASEINCOMPLETE` | `case` misses values | add the missing items or a `default` |
| `BLKSEQ` | = in a sequential block | <= in `always_ff` |
| `MULTIDRIVEN` | driven from two blocks | exactly one block |
| `DECLFILENAME` | file name differs from module | one module per file |

# Simulation and Waveforms

```
# SystemVerilog testbench with Icarus Verilog
iverilog -g2012 -o testbenches/verilog/tick_gen_tb.vvp \
    rtl/tick_gen.sv testbenches/verilog/tick_gen_tb.sv
(cd testbenches/verilog && vvp tick_gen_tb.vvp)

# cocotb testbench with Icarus Verilog, then with Verilator
(cd testbenches/cocotb && python3 tick_gen_tb.py)
(cd testbenches/cocotb && SIM=verilator python3 tick_gen_tb.py)

# Waveforms
gtkwave testbenches/verilog/tick_gen_tb.vcd
surfer testbenches/cocotb/sim_build/tick_gen.fst
```

# A Quick Synthesis Check with Yosys

```
yosys -p "read_verilog -sv rtl/tick_gen.sv; synth -top tick_gen; stat"
```

- Flip-flops (`$_DFF_*`): as many as the **registers you intended**
- Latches (`$_DLATCH_*`): **none**
- Tick generator (`DIV = 10`): 17 cells, 4 × `$_DFFE_PN0P_` and 1 × `$_DFF_PN0_`
  - `PN0P`: positive clock, active-low reset to 0, active-high enable

# Live Demo: Tick Generator

```
mkdir -p /foss/designs/sv_exercises/tick_gen/{rtl,testbenches/verilog,testbenches/cocotb}
cd /foss/designs/sv_exercises/tick_gen
```

- Save `rtl/tick_gen.sv` and both testbenches (Sections 2.5.7, 2.6)
- **Lint**, **simulate** (SystemVerilog and cocotb), look at the **waves**, **synthesize**
- Build in a bug (`COUNT_LAST - 1`): do both testbenches catch it?

**Exercise** (Section 2.7.6): the tick generator, then a PWM generator with a SystemVerilog and a cocotb testbench, and a sequence detector FSM.

# Hands-On: The Template Counter

```
cd /foss/designs
git clone https://github.com/iic-jku/ihp-sg13cmos5l-ams-chip-template.git
cd ihp-sg13cmos5l-ams-chip-template/macros/counter
```

- `macros/counter`: 8-bit counter, synchronous reset, enable, wrap at `CTR_MAX`
- Lint and simulate it **by hand**: what does each testbench check?
- Rewrite it in the **two-process style**, build in a bug, check the exit status
- What is behind the **make targets**?
  - `make help`, `make -n sim-rtl-verilog`

AMS chip template

**Exercise** (Section 2.8): three exercises with the template counter.

# Guidelines and Summary



# HDL Coding Guidelines

*"A consistent naming and schematic drawing style, as well as VHDL/Verilog coding scheme, is a huge help in avoiding errors and increasing productivity."* (Circuit Designer's Etiquette)

- **Lint** with `-Wall`, fix every warning
- `_i`, `_o`, `_io`, active low: `n` before it (`rst_ni`)
- `logic` everywhere, explicit ports
- `localparam`, parameters in UPPERCASE
- **Reset** every control register
- **One module per file**, same name
- `` `default_nettype none `` ... `wire`
- `always_comb`, never `always @(*)`
- Explicit **widths** and conversions
- `=` in `always_comb`, `<=` in `always_ff`

From lowRISC: `_d`/`_q`, `_e`/`_t` types, `St` states, `unique case` + `default`, named connections

# Pitfalls for VHDL Designers

| Pitfall | What happens | Fix |
|---|---|---|
| expecting `reg` to be a register | storage follows from the block | use `logic` |
| `=` in `always_ff` | order-dependent, simulation differs from synthesis | `<=` in `always_ff` |
| missing `else` or `default` | a latch | defaults first |
| `Clk` and `clk` | two different signals | lowercase snake_case |
| a typo in a name | an implicit 1-bit wire | `` `default_nettype none `` |
| mismatched widths | silent truncation, a lost carry | sized literals, lint |
| `==` with `x` in a testbench | can be `x`, the check passes | `!==`, `$isunknown()` |
| `initial` for reset values | not synthesizable for an ASIC | reset in `always_ff` |
| driving on the active edge | races, simulators differ | drive and check on the falling edge |
| `$error` in Icarus Verilog | exit status 0 | `$fatal(1, ...)` |

# Summary

- An HDL describes **concurrent hardware**: think hardware first
- Register + next-state logic: **one clock period** to compute
- Alternative HDLs end as **(System)Verilog**: **read** Verilog, **write** SystemVerilog
- Your **two-process style** maps directly: `always_ff` copies `_d` into `_q`, `always_comb` holds the defaults and all decisions
- `logic`, `=` in `always_comb`, `<=` in `always_ff`, sized literals, **lint everything**
- **Self-checking** testbenches that set the **exit status**: `$fatal`, `get_results()`

# Exercises

1. Loops in Software and in Hardware (2.2.1)
2. A 4:1 Multiplexer in SystemVerilog (2.5.4)
3. Let the Linter Find the Bugs (2.5.8)
4. The Controller as a Mealy FSM (2.5.9)
5. Modernize a Verilog-2001 Module (2.5.13)
6. Lint, Simulate, and Synthesize the Tick Generator (2.7.6)
7. PWM Generator with a SystemVerilog Testbench (2.7.6)
8. PWM Generator with a cocotb Testbench (2.7.6)
9. Sequence Detector FSM (2.7.6)
10. Lint and Simulate the Counter by Hand (2.8)
11. The Counter in the Two-Process Style (2.8)
12. What Is Behind the Make Targets? (2.8)



Chapter 2



JOHANNES KEPLER
UNIVERSITY LINZ